

Autograd vs. Parameter-Shift Gradient for Tensor Ring VQE: Investigation and Findings

TREV Project

April 12, 2026

Abstract

We investigate reverse-mode automatic differentiation (autograd) through differentiable SVD for computing VQE gradients in the TREV tensor ring simulator. Autograd produces exact gradients ($\cos = 1.000$ vs. parameter-shift) for circuits without lossy SVD truncation, but degrades ($\cos \approx 0.66$) for deep transpiled circuits where many two-qubit gates involve lossy truncation. We trace the root cause to the non-smoothness of truncated SVD and show that no existing tensor network library has solved this for 50+ chained SVDs. All successful approaches avoid the problem entirely.

1 Background

TREV represents quantum states as tensor rings with bond dimension χ . Each two-qubit gate requires SVD to split the merged two-site tensor:

$$M = U\Sigma V^\dagger \approx U_k \Sigma_k V_k^\dagger, \quad k = \chi \quad (1)$$

where $M \in \mathbb{C}^{2\chi \times 2\chi}$ and only top- k singular values are retained. The parameter-shift rule computes exact gradients via $2P$ forward evaluations. Autograd requires only 1 forward + 1 backward pass.

2 Differentiable SVD Backward

Our `diff_svd` implementation combines:

F/G split formula (TensorKit convention).

$$\frac{\partial L}{\partial A} = U \left[(a_U + a_V) \odot F + (a_U - a_V) \odot G + \text{diag}(\bar{\Sigma}) \right] V^\dagger \quad (2)$$

where $F_{ij} = 1/(s_j - s_i)$, $G_{ij} = 1/(s_j + s_i)$, $a_U = (U^\dagger \bar{U} - \bar{U}^\dagger U)/2$.

Lorentzian broadening for degenerate singular values: $1/x \approx x/(x^2 + \epsilon)$.

Wan-Zhang complex correction [?].

$$\frac{\partial L}{\partial A} += U \text{diag} \left(\frac{L_i - L_i^*}{2s_i} \right) V^\dagger, \quad L_i = (U^\dagger \bar{U})_{ii} \quad (3)$$

Gauge projection. The tensor ring stores $q_0 = U_k \Sigma_k$ and $q_1 = V_k^\dagger$ as separate cores. Their independent contraction gradients violate gauge invariance: $\text{Im}(\text{diag}(U_k^\dagger \bar{U}_k)) + \text{Im}(\text{diag}(V_k^\dagger \bar{V}_k)) \neq 0$. We project out the gauge-dependent component before the F/G formula.

3 Results

3.1 Per-SVD Accuracy

Matrix type	k/r	$\cos(\nabla_{\text{ad}}, \nabla_{\text{FD}})$
Full rank (random)	4/8	0.9999
Rank-2 (circuit MPS)	4/8	0.9554
Rank-2, no truncation	4/4	0.9999

3.2 Circuit-Level Accuracy

Circuit	#2q gates	Lossy truncations	$\cos(\nabla_{\text{ad}}, \nabla_{\text{ps}})$
HEA $N=6, \chi=4$	12	0	1.000
HEA $N=9, \chi=8$	18	0	1.000
Ring $N=9, \chi=2$	9	1–2	0.990
Ring $N=9, \chi \geq 4$	9	0	1.000
TSP QAOA $\chi=4$	52	~ 18	0.661
TSP QAOA $\chi=8$	52	~ 18	0.593

3.3 Root Cause: Non-Smoothness of Truncated SVD

The expectation value $E(\theta)$ with SVD truncation is non-smooth. Finite differences at varying step sizes diverge:

Step size ϵ	FD estimate $\frac{f(\theta+\epsilon) - f(\theta-\epsilon)}{2\epsilon}$
10^{-1}	−65
10^{-2}	+124
10^{-3}	+7175
10^{-4}	−46890
10^{-5}	+925614

This divergence occurs for **both** `build_tensor` and `_build_tensor_diff`, confirming the non-smoothness is intrinsic to SVD truncation. At the test point, 19 of 52 gates have exactly degenerate singular values at the truncation boundary ($s_k = s_{k+1} = 0$). The parameter-shift rule evaluates at $\theta \pm \pi/2$, averaging over cusps. Autograd computes the local subgradient, hitting cusps directly.

4 Approaches Attempted

4.1 Successful

1. **Wan-Zhang complex correction** [?]: Per-SVD accuracy $0.78 \rightarrow 0.9999$.
2. **cfloat forward matching**: Eliminated 3×10^{-4} EV discrepancy.

3. **Gate support:** `ParameterTwoQubitGate`, `ParameterMultiOneQubitGate`, wrap-around handling.
4. **Gauge projection:** Corrected gradient magnitudes from $\sim 3\times$ mismatch to $\sim 1.0\times$.

4.2 Unsuccessful

1. **Taylor polynomial F-matrix** [?]: Amplifies degenerate pairs; $\cos = 0.02$ for deep circuits.
2. **Francuz truncation correction** [?]: Three implementations all double-counted with existing $r \times r$ F/G cross-terms.
3. **Gradient checkpointing:** Identical results at all segment sizes (systematic, not stochastic error).
4. **Full-rank SVD + external slicing:** Mathematically identical backward.
5. **Symmetric S absorption:** Changes forward; NaN from $\sqrt{0}$.
6. **Adjoint method:** 1-qubit $\cos = 0.95$; 2-qubit $\cos = 0.40$.
7. **Real-form SVD:** Paired degenerate SVs cause equivalent problems.
8. **Float64 backward:** Already implemented; no improvement.

5 Literature Survey

Library	Strategy	Effective #SVD	Validated
peps-torch [?]	Fixed-point differentiation	~ 1	Yes
variPEPS	Fixed-point + Francuz	~ 1	Yes
TensorCircuit [?]	Identity init (avoids truncation)	0	No
quimb [?]	Exact contraction (no SVD)	0	N/A
TREV (this work)	Direct SVD backward chain	Up to 52	Partial

No existing library has demonstrated correct gradients through 50+ chained truncated SVDs. Successful approaches avoid the problem: peps-torch collapses iterations via the implicit function theorem (~ 1 effective SVD), TensorCircuit avoids truncation, quimb uses exact contraction.

6 Conclusions

1. Autograd matches parameter-shift exactly ($\cos = 1.000$) when truncation is lossless.
2. For deep circuits with lossy truncation, autograd gives $\cos \approx 0.66$ due to non-smoothness.
3. This is a fundamental mathematical limitation, not a code bug.
4. **Future directions:** fixed-point differentiation, smooth truncation, or sufficient χ to avoid lossy truncation.

References

- [1] Z.-Q. Wan, S.-X. Zhang, “Automatic differentiation for complex valued SVD,” arXiv:1909.02659 (2019).
- [2] A. Francuz, N. Schuch, B. Vanhecke, “Stable and efficient differentiation of tensor network algorithms,” *Phys. Rev. Research* **7**, 013237 (2025).
- [3] Y. Song, N. Sebe, “Why approximate matrix square root outperforms accurate SVD in global covariance pooling?” *ICCV* (2021).
- [4] H.-J. Liao, J.-G. Liu, L. Wang, T. Xiang, “Differentiable programming tensor networks,” *Phys. Rev. X* **9**, 031041 (2019).
- [5] S.-X. Zhang et al., “TensorCircuit: a quantum software framework for the NISQ era,” *Quantum* **7**, 912 (2023).
- [6] J. Gray, “quimb: a python library for quantum information and many-body calculations,” *JOSS* **3**, 819 (2018).